

# Blowing in the Wind

Xiaoming Wei, Ye Zhao, Zhe Fan, Wei Li, Suzanne Yoakum-Stover and Arie Kaufman

{wxiaomin,yezhao,fzhe,liweili,suzi,ari}@cs.sunysb.edu}

Center for Visual Computing (CVC) and Department of Computer Science  
Stony Brook University  
Stony Brook, NY 11794-4400

---

## Abstract

*We present an approach for simulating the natural dynamics that emerge from the coupling of a flow field to light-weight, mildly deformable objects immersed within it. We model the flow field using a Lattice Boltzmann Model (LBM) extended with a subgrid model and accelerate the computation on commodity graphics hardware to achieve real-time simulations. We demonstrate our approach using soap bubbles and a feather blown by wind fields, yet our approach is general enough to apply to other light-weight objects. The soap bubbles illustrate Fresnel reflection, reveal the dynamics of the unseen flow field in which they travel, and display spherical harmonics in their undulations. The free feather floats and flutters in response to lift and drag forces. Our single bubble simulation allows the user to directly interact with the wind field and thereby influence the dynamics in real time.*

Categories and Subject Descriptors (according to ACM CCS): I.3.5 [Computer Graphics]: Computational Geometry and Object Modeling; I.3.7 [Computer Graphics]: Three-Dimensional Graphics and Realism

---

## 1. Introduction

The demand for increasingly realistic visual simulations of complex phenomena is tremendous. The ultimate goal is to bring the graphics and animations alive so that trees are blown in the wind, flags flap, fluids flow, and hair flies. Faithful simulation of these effects involve three essential elements: a flow field, object model(s), and the coupling between them. The approach presented in this paper is motivated by the recognition that to support interactive simulations or respond to events occurring in the scene, each of these three elements must be computed in real-time. Furthermore, exhibiting realistic dynamics under the broadest range of conditions, requires the use of models that are as physically accurate as possible. And, since boundary conditions largely define a flow field while also playing an extremely important role in object dynamics and coupling, it is essential to be able to handle complex shaped and moving boundaries.

In this paper, we present an approach for modeling the blowing effect of a wind field on light-weight deformable objects. We model the wind field using the Lattice Boltzmann Model (LBM), which is a relatively new computa-

tional fluid dynamics (CFD) method<sup>9, 28</sup>. Compared to other CFD methods, LBM has the advantages of being easy to implement, parallelizable, and able to handle complex and moving boundaries<sup>11, 12</sup>. The objects we model include soap bubbles with spherical harmonics representing their deformation and oscillation and a free feather with thin strip flutter dynamics. We chose these objects to illustrate our approach mainly because they are visually interesting, however our method can be applied to any light-weight object. By considering only light-weight objects, we are able to produce realistic-looking simulations without having to model the back-coupling of the object on the flow field. We compute both the wind field and the object dynamics in real time which allows the simulations to be interactive. For this purpose, we accelerate parallel LBM computation on commodity graphics hardware in a manner similar to that of Li et al.<sup>5</sup>.

The remainder of the paper is organized as follows. In the next section, we provide an overview related work describing alternate approaches to flow modeling and object interactions. The LBM used to model the wind fields in our simulations is described in Section 3. In Sections 4 and 5,

we present the models used for the soap bubbles, the feather, and their interaction with the wind field. The implementation of hardware acceleration and simulation performance are described in Section 6.

## 2. Related Work

Over the past decade, researchers have used a variety of approaches for modeling the interaction of wind with leaves, trees, grass, and human hair<sup>21, 23, 24, 25, 26, 27</sup>. Specifically, Wejchert and Haumann<sup>21</sup> developed an aerodynamic model for simulating the detachment and falling of leaves from trees in a wind field built from four basic flow primitives. Shinya and Fournier<sup>24</sup> animated trees and grass blown by complex flow fields by modeling the wind field in Fourier space and then converting to a time-varying force field. Using similar methods, Stam<sup>23</sup> animated trees in turbulent winds. More recently, Perbet and Cani<sup>25</sup> simulated the movement of the grass in a wind field which was generated by combining procedural primitives and an additional time-varying stochastic component. In contrast to the above, we use a CFD approach for modeling the wind field and only consider the motion of free, light-weight objects. However, our wind model and force evaluation method could also be used with these grass and tree models.

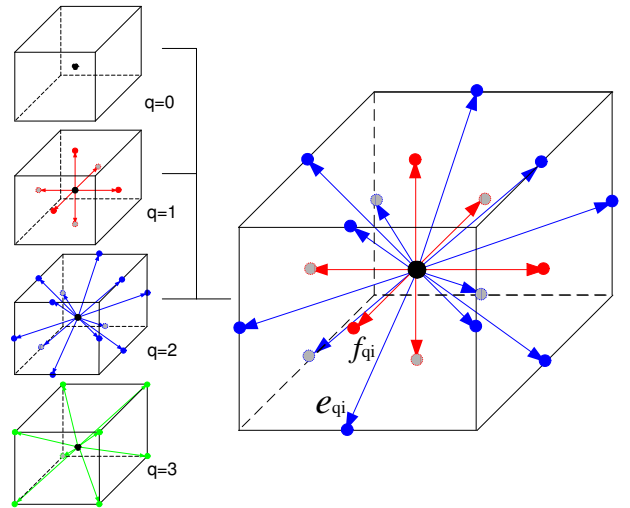
In recent years, the application of CFD methods for solving the Navier-Stokes (NS) equations by graphics and visualization researchers has led to significant developments in the visual simulation of gases and fluids. In 1997 Foster and Metaxas<sup>2</sup> presented a full 3D finite difference solution to simulate the turbulent rotational motion of gas. In 1999 Stam<sup>1</sup> devised a fluid solver using semi-Lagrangian advection schemes and implicit solvers resulting in an unconditionally stable method that produces compelling simulations of turbulent flows. Their method can also achieve real-time speed on a low-resolution grid. Unfortunately, numerical dissipation causes the turbulence to decay too rapidly. Fedkiw et al.<sup>3</sup> addressed this shortcoming by feeding energy back into vortices using vortex confinement methods. Recently, a quite different CFD method, the Lattice Boltzmann Model, was introduced to the graphics community by Wei et al. to model a dynamic wind field and moving solid boundary objects in a simulation of fire and smoke<sup>4, 6</sup>. We have chosen to take up this alternate approach because it provides physical accuracy, particularly with complex boundary geometries<sup>10</sup>, while being quite simple to implement and amenable to hardware acceleration.

## 3. Simulating Wind

We begin this section with a brief review of LBM.

### 3.1. Lattice Boltzmann Model

The LBM is a relatively new method for CFD, inspired by cellular automata, that models Boltzmann particle dynamics



**Figure 1:** The D3Q19 lattice geometry. On the left side are shown the four sub-lattices that are defined in a 3D grid and on the right is the combination of sub-lattices 0, 1, and 2. The packet distribution  $f_{qi}$  is associated with the link corresponding to the  $\mathbf{e}_{qi}$  velocity vector.

on a lattice<sup>28</sup>. In the case of a fluid, for example, the Boltzmann equation expresses how the average number of flow particles with a given velocity changes between neighboring lattice sites due to inter-particle interactions and ballistic motion. In the LBM, the variables associated with each lattice site are velocity distributions, or “packet distributions,” which represent the probability of the presence of a flow particle with a given velocity. The set of velocity vectors in the model is discrete being defined by the number, orientation, and length of lattice links. Particles stream along lattice links from one node to another synchronously in discrete time steps. Between streaming steps, they undergo collision. The Bhatnager, Gross, Krook (BGK) model is typically used to represent the effect of these collisions as a statistical redistribution of momentum which locally drives the system toward equilibrium<sup>9</sup>. As in kinetic theory, the collisions conserve mass, momentum, and in thermal models, energy. Unlike iterative solvers that compute a steady state, the LBM is an explicit time stepping procedure for modeling time varying flows. In the limit of zero time step and lattice spacing, the LBM flow model yields the Navier-Stokes equation for incompressible fluid flow. As a kind of explicit finite difference method, the stability of LBM for flow simulation is limited to flows with low Mach number (i.e. flow velocities small compared to the speed of sound). However, the LBM can be enhanced with a subgrid model representing small scale energy damping in order to achieve faster flows with high Reynolds numbers<sup>7</sup>.

As illustrated in Figure 1, a 3D LBM lattice may be rep-

resented as a combination of four sub-lattices: Sub-lattice 0 consists of the center cell with zero velocity; sub-lattice 1 includes six nearest neighbor links; sub-lattice 2 has twelve second-nearest neighbor links; and sub-lattice 3 consists of eight third-nearest neighbor links. For our LBM model we employ the so called D3Q19 lattice which is a 3D lattice with 18 links representing 19 velocity vectors (including the zero velocity), consisting of sub-lattices 0, 1 and 2. Stored at each lattice node are 19 packet distributions associated with the 19 velocity vectors. We denote these as  $f_{qi}$  where  $q$  identifies a particular sub-lattice and  $i$  identifies a particular velocity vector within that sub-lattice. The macroscopic fluid density,  $\rho$ , and velocity,  $\mathbf{u}$ , are computed from the packet distributions as follows:

$$\rho = \sum_{qi} f_{qi} \quad (1)$$

$$\mathbf{u} = \frac{1}{\rho} \sum_{qi} f_{qi} \mathbf{e}_{qi} \quad (2)$$

Using the BGK single-time-relaxation model, the Boltzmann dynamics can be represented as a two-step process of collision and ballistic streaming. Taken together they can be represented as

$$f_{qi}(\mathbf{x} + \mathbf{e}_{qi}, t + 1) - f_{qi}(\mathbf{x}, t) = -\frac{1}{\tau} (f_{qi}(\mathbf{x}, t) - f_{qi}^{eq}(\rho, \mathbf{u})) \quad (3)$$

where the local equilibrium packet distribution is given by

$$f_{qi}^{eq}(\rho, \mathbf{u}) = \rho (A_q + B_q (\mathbf{e}_{qi} \cdot \mathbf{u}) + C_q (\mathbf{e}_{qi} \cdot \mathbf{u})^2 + D_q \mathbf{u}^2) \quad (4)$$

The constant  $\tau$  represents the relaxation time scale which determines the viscosity of the flow, while  $A_q$  through  $D_q$  are constant coefficients specific to the chosen lattice geometry. The equilibrium packet distribution comes in as a consequence of the BGK collision model. It is a local packet distribution whose value depends on conserved quantities - the macroscopic mass  $\rho$  and momentum  $\rho \mathbf{u}$ . Since the calculation of Equations 1 to 4 only requires local packet distribution values, their acceleration on graphics hardware is efficient.

### 3.2. Subgrid Model

Assuming a unit time step and lattice spacing, the kinematic shear viscosity  $\nu$  in the LBM is related to the relaxation parameter  $\tau$  as:

$$\nu = \frac{2\tau - 1}{6} \quad (5)$$

To reach high Reynolds number flows requires  $\nu$  to be as small as possible which implies decreasing  $\tau$  toward  $1/2$ . Unfortunately, this leads to numerical instability. The roots of this instability lie in the inability of the LBM to represent flow dynamics on scales smaller than the lattice spacing<sup>7</sup>. Without these, the energy transfer from larger scales to smaller scales that is needed to stabilize the model can not occur. One way to solve this problem is to combine a traditional subgrid model with the LBM. Note that such subgrid

models have nothing to do with grid refinement. What the subgrid model does is to represent the physical effects that the small scale (i.e. unresolved) motion has on the LBM-scale (i.e. resolved) motion. In particular, the Smagorinsky subgrid model represents the influence of the unresolved scales by adding a positive eddy viscosity to the kinematic shear viscosity. In doing so, the single-time-relaxation approximation is abandoned and the value of  $\tau$  is allowed to vary over the lattice. The key point, as we explicitly show below, is that the local variation of  $\tau$ , which depends on the value the local strain tensor, is computed for a given node only using neighboring nodes. Following the work of Hou et al.<sup>7</sup>, the procedure goes as follows:

We compute the nonequilibrium momentum tensor  $\Pi_{m,n}^{(1)}$ , where  $m$  and  $n$  represent cartesian components.

$$\Pi_{m,n}^{(1)} = \sum_{qi} e_{qi,m} e_{qi,n} (f_{qi} - f_{qi}^{eq}) \quad (6)$$

The magnitude of the local strain tensor  $S$ , is then given by

$$|S| = \frac{-\nu + \sqrt{\nu^2 + 18C^2 \sqrt{\Pi_{m,n}^{(1)} \Pi_{m,n}^{(1)}}}}{6C^2} \quad (7)$$

The Smagorinski eddy viscosity, given by

$$\nu_t = C^2 |S| \quad (8)$$

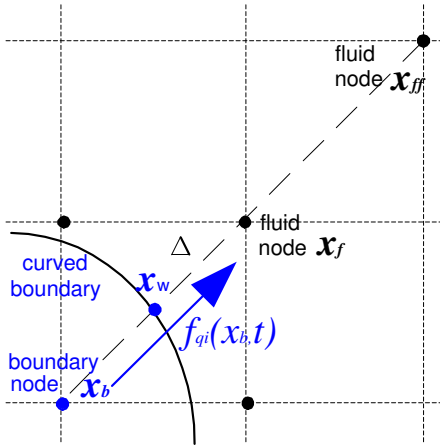
is then added to the shear viscosity to locally adjust the value of  $\tau$  as:

$$\tau_{new} = 3(\nu + C^2 |S|) + \frac{1}{2} \quad (9)$$

where  $C$  is the Smagorinsky constant (typically 0.04). Since evaluating Equation 9 only involves local packet distribution values, the model retains its parallelizability.

### 3.3. Boundary Conditions

Boundary conditions in the LBM may take several forms. The simplest are periodic boundary conditions for which the out-going packet distribution value re-enters the grid along its original velocity direction on the opposite face of the lattice volume. To represent out-flow conditions corresponding to flow that simply leaves the volume, the packet distributions of the nodes just inside the boundary are simply copied to fictitious nodes just outside the boundary. For straight boundaries, the so called no-slip boundary condition which expresses the requirement that the tangential component of the fluid velocity along the boundary be zero, are implemented with a simple bounce-back rule. In this case, the out-going packet distribution value re-enters the grid at the same node, but associated with the opposite velocity. Another commonly used boundary condition applied to flow volume boundaries where the flow is largely parallel to the boundary and the boundary itself cuts through the nodes, is a kind of bounce-forward condition. The rule is to copy the mirror image of the packet distributions of the nodes that on



**Figure 2:** A slice of a regularly spaced lattice and a curved wall boundary.

the fluid side of the boundary to those on the solid side. The net effect on the nodes in between (i.e., the ones directly on the boundary) is to null out the component of the velocity normal to the boundary. This bounce-forward rule is mainly appropriate for flow volume faces that are far from the fluid region of interest. Discussions of other kinds of boundary conditions for which pressure or velocity are specified may be found in the literature (see Maier et al.<sup>29</sup> and references therein).

For no-slip boundaries that can move, or involve complex geometries, or even that are straight but do not lie on the lattice nodes, the bounce back rule has been substantially improved by Mei et al.<sup>11</sup>. Because of its importance, we explicitly state it here. As indicated in Figure 2, in order to compute the packet distribution for a node,  $x_f$ , in the fluid next to the boundary we need to know the packet distributions of its neighbors. Thus, a fictitious packet distribution,  $f_{qi}(\mathbf{x}_b, t)$ , is defined on the node  $\mathbf{x}_b$  which lies just inside the object boundary. The fraction of the link that is intersected by the boundary in the fluid region is denoted by  $\Delta = |\mathbf{x}_f - \mathbf{x}_w| / |\mathbf{x}_f - \mathbf{x}_b|$ , where  $0 \leq \Delta \leq 1$ . The post collision value of  $f_{qi}(\mathbf{x}_b, t)$  is given by:

$$f_{qi}(\mathbf{x}_b, t) = (1 - \chi)f_{qi}(\mathbf{x}_f, t) + \chi f_{qi}^*(\mathbf{x}_b) + 6A_q \rho \mathbf{e}_{qi} \cdot \mathbf{u}_w \quad (10)$$

where,

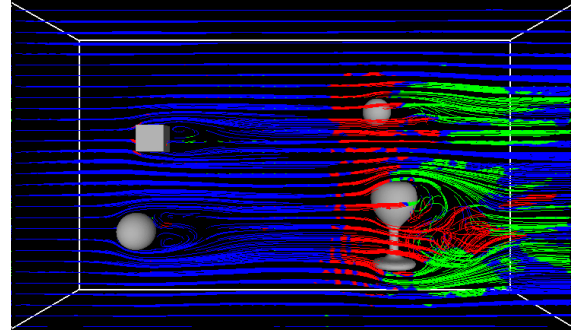
$$f_{qi}^*(\mathbf{x}_b) = \rho(A_q + B_q \mathbf{e}_{qi} \cdot \mathbf{u}_{bf} + C_q (\mathbf{e}_{qi} \cdot \mathbf{u}_f)^2 - D_q (\mathbf{u}_f)^2) \quad (11)$$

and for  $\Delta \geq 1/2$ ,

$$\mathbf{u}_{bf} = (1 - \frac{3}{2\Delta})\mathbf{u}_f + \frac{3}{2\Delta}\mathbf{u}_w \quad \text{and} \quad \chi = \frac{2\Delta - 1}{\tau + 1/2} \quad (12)$$

while for  $\Delta < 1/2$ ,

$$\mathbf{u}_{bf} = \mathbf{u}_{ff} \quad \text{and} \quad \chi = \frac{2\Delta - 1}{\tau - 2}. \quad (13)$$



**Figure 3:** Streamlines originating from a vertical plane that cuts through the most active flow region. (See Color Plate 3 in the color section.)

Note that  $\mathbf{u}_{bf}$  represents the virtual speed of the boundary node  $\mathbf{x}_b$  in terms of the fluid velocity,  $\mathbf{u}_f$ , at node  $f$ , and the boundary velocity,  $\mathbf{u}_w$ , at  $x_w$ . Thus we see that this rule (which has second order accuracy and good stability) accounts for both arbitrarily shaped and moving boundaries.

Using the above no-slip boundary condition rule, we are also able to implement flow inlets in an interesting way. Imagine a tube attached to the flow volume wall. We could force fluid through the tube and into the volume by sliding a plunger through the tube. If the tube were also part of the LBM flow volume, we would model this process by applying bounce-back rule to the tube and the moving plunger where the plunger velocity would play the role of  $\mathbf{u}_w$ . Unfortunately, the amount of fluid that we could push into the volume by this method would be limited by the extent and volume of the tube. However, we can avoid this complication and achieve the same effect by considering the plunger just as it reaches the flow volume wall. We apply the bounce-back rule on the plunger, which implies that it is moving, but we do not actually move the plunger in the next time step. Repeatedly pushing the plunger without actually moving it, would have the effect of a propeller. We have used this kind of propeller boundary to create inlets in many of our simulations.

### 3.4. Wind Field Generation

To facilitate the creation of a wind field with interesting vortex structures, we built an interactive GUI with controls that allow us to position various obstacles in the flow. For example, Figure 3 and Color Plate 3 show snapshots of the flow structure that results in our LBM model when two balls, a cube, and an odd-shaped object (goblet) are placed in the flow volume. Prior to their insertion, a horizontal flow (from left to right) was established by setting a velocity boundary condition on the left vertical face of the flow volume. In addition, we used periodic boundary conditions on the left and right vertical faces of the flow volume. On the remaining



faces of the flow volume we used a simple bounce-forward rule, as described in the previous section. As can be seen from the figure, complex flows can be generated by placing obstacles in the flow to create interesting vortices.

### 3.5. Coupling Wind to Objects

To simulate the effect of wind blowing on an object, the coupling of the wind to the object must be modeled. For the simulations presented here, we achieve this by evaluating the force exerted by the wind. Various approaches to LBM force evaluation have been presented in the CFD literature (see <sup>11</sup> and references therein). Of these, the momentum-exchange method is probably the simplest to implement and most computationally efficient. The force on a boundary node of the object is computed as the net momentum exchanged with neighboring fluid nodes. The total force is then obtained by summing the contributions associated with all the object boundary nodes. In other words, the force exerted by the flow on an object results from the change in momentum of the flow particles that collide with the object.

The momentum-exchange method clearly takes into account the modification of the flow due to the presence of the object. This is overkill for light-weight objects like the soap bubbles and feather that we have chosen to model. For objects of this kind, it is reasonable to assume that the flow is unperturbed by the object but not vice versa. Under this assumption, the object presents no real boundary to the flow and the momentum exchange model cannot be applied directly. We take therefore, a different approach by considering the “virtual force” that would appear if we imagined the object surface to be embedded in the flow. From this perspective, the effect of the flow is two-fold: It advects the object along the flow stream lines and, as those stream lines converge and diverge, it causes deformation. The former exerts no force on the object in the moving frame of the object/fluid. Thus, we compute our virtual force from the change in velocity of the flow at the object surface in the object’s moving frame of reference.

### 3.6. Computational Procedure

The computation procedure underlying our simulations involves the following series of steps:

0. Initialize LBM. Specifically set all packet distribution values equal to the equilibrium distribution value (Equation 4) with zero velocity and constant density.
1. Perform collision and streaming (Equation 3).
2. Apply boundary condition rules.
3. Compute macroscopic density (Equation 1) and velocity (Equation 2).
4. Adjust relaxation time according to subgrid model (Equation 9).
5. Compute object physics (forces and deformation).
6. Advect object.

7. Render scene.
8. Return to step 1.

In interactive simulations, when the user modifies or moves boundary objects, a GUI listener directly updates the boundary geometry data. This means that the LBM calculations always use the current boundary configuration, although the packet distributions at nodes that have changed from fluid to boundary or vice versa may not be immediately updated. In other words, a fluid node that suddenly becomes a boundary node may be used in step 1 before its packet distribution has been properly updated. Fortunately, provided that objects in the LBM move slowly compared to the LBM speed of sound, such errors are small and heal rapidly without having adverse effects on the simulation.

## 4. Soap Bubble Simulations

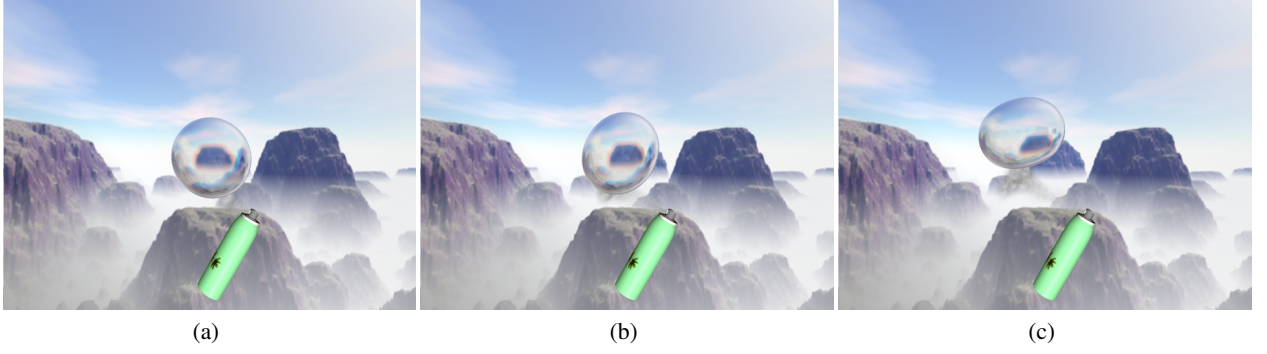
Soap bubbles do several amazing things that make them fascinating to watch. They stunningly illustrate Fresnel reflection and refraction. In large swarms they reveal the invisible structure of the flow field in which they travel. And, larger bubbles display the spherical harmonics in their undulations. The soap bubble simulations that we present to illustrate our method display all of these features (except refraction) and the observed dynamics appears as a natural consequence of the interaction between physical models.

Much of the work on soap films, bubbles, and foams focuses on the properties of bubble clusters (see <sup>14, 15, 16</sup> and references therein). In contrast, we focus on the advection and deformation of individual bubbles influenced by an active wind field, ignoring collisions and other bubble-bubble interactions. We consider the soap film thickness to be uniform and idealize the bubble shape dynamics in terms of the spherical harmonics which represent the normal modes of the sphere.

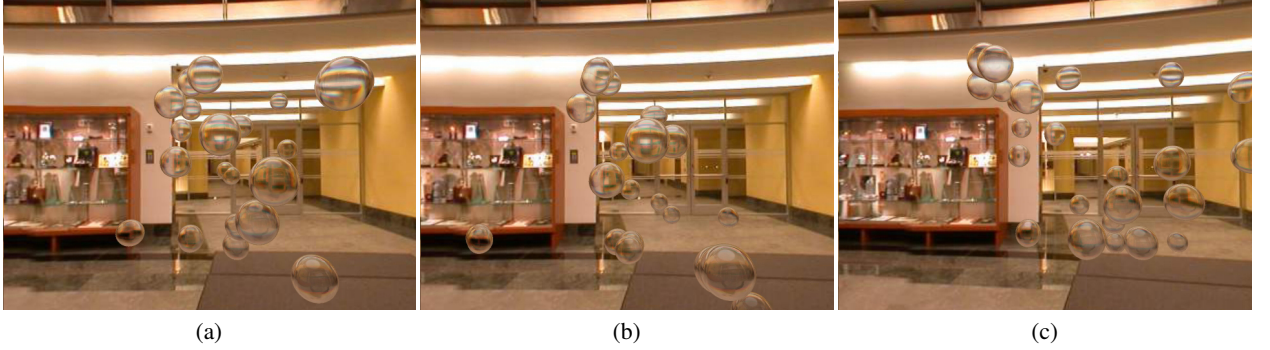
We represent the bubble dynamics in terms of three independent phenomena - advection, body force, and deformation. The bubble is advected by the flow according to the local velocity, at its center of mass, obtained from the LBM. The body force, due to gravity, modifies the advection velocity by the addition of a vertical component. And, the bubble deformation is caused by its being squeezed along the direction of the flow. The magnitude of this squeeze is made proportional to the gradient of the flow velocity in the bubble’s frame of reference along the direction of the flow.

In general, any square-integrable function  $r(\theta, \phi)$ , such as the one representing the bubble surface, can be uniquely represented as a superposition of spherical harmonics,  $Y_l^m(\theta, \phi)$ , where  $\theta \in [0, \pi]$  is the angle with respect to the  $\mathbf{z}$ -axis and  $\phi \in [0, 2\pi]$  is the angle measured from the  $\mathbf{x}$ -axis in the  $x-y$  plane:

$$r(\theta, \phi) = \sum_l \sum_m \rho_{l,m} Y_l^m(\theta, \phi) \quad (14)$$



**Figure 4:** Soap bubble deformation and dynamics in response to the spray from the air-can. (See Color Plate 1 in the color section.)



**Figure 5:** A sequence of snapshots taken from our multi bubble animation, as we move the viewpoint position from right to left. (See Color Plate 2 in the color section.)

Inverting allows the expansion coefficients to be computed from the shape:

$$\rho_{l,m} = \int_0^{2\pi} d\phi \int_0^\pi r(\theta, \phi) Y_l^m \sin(\theta) d\theta. \quad (15)$$

For example, the shape of a bubble slightly squeezed along the  $z$ -axis has the following representation in terms of spherical harmonics:

$$r(\theta, \phi) = (r_0 - \frac{\delta^2}{6})\sqrt{4\pi} Y_0^0(\theta, \phi) - \frac{2\delta^2}{3r_0}\sqrt{\frac{\pi}{5}} Y_2^0(\theta, \phi) \quad (16)$$

where the coefficient  $r_0$  is the unperturbed radius and  $\delta$  is the magnitude of the deformation. The development of this expansion relies on the small-amplitude condition that  $\delta \ll r_0$ , hence “slightly” squeezed. Note that in the case of no deformation,  $\delta = 0$ , we recover the unperturbed spherical bubble shape since  $Y_0^0(\theta, \phi) = 1/\sqrt{4\pi}$ .

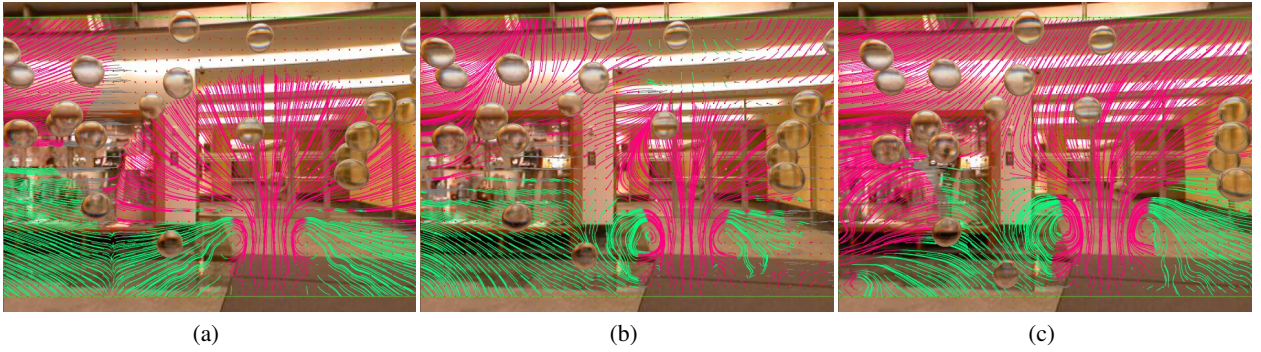
Each spherical harmonic in the superposition represents a natural vibrational mode of the sphere. In terms of these it is easy to represent the shape oscillations:

$$r(\theta, \phi, t) = \sum_l \sum_m \rho_{l,m} Y_l^m(\theta, \phi) e^{-\beta_l t} \quad (17)$$

The mode frequencies and their decay rates, given by the real and imaginary parts of  $\beta$  respectively, depend on the physical properties of the soap film<sup>17</sup>. In general terms however, the mode frequencies increase with  $l$ , and modes having the same  $l$  but different  $m$  oscillate with the same frequency. For the bubble, for which the volume does not change as it oscillates, the spherically symmetric  $l = 0$  “breathing” mode does not occur (i.e.  $\beta_0 = 0$ ). The next higher order mode,  $l = 1$ , the dipole oscillations, also do not appear since the center of mass of a freely oscillating bubble (i.e., subject to no external forces) does not move. Thus, we find that the lowest order allowed mode for the free bubble is  $l = 2$ .

As can be seen from Equation 16, the  $l = 2$  mode is clearly involved in the oscillations of the slightly squeezed bubble. Although relaxing the small amplitude approximation that led to that equation does introduce higher order modes, they appear with smaller amplitudes. Hence, we capture most of the oscillation dynamics by considering only the  $l = 2$  mode in our simulations. Its frequency and decay rate are simulation parameters with values chosen to approximate the behavior of real bubbles observed in the laboratory.

Although a standard ray tracing algorithm could be used



**Figure 6:** Several bubbles blown by a flow as large vortices develop. Streamlines show the flow pattern originating from a plane cutting through the most active flow region. (See Color Plate 4 in the color section.)

for rendering bubbles, attempting to capture the thin film reflection and refraction effects would make it complicated and slow. Instead we use Cube Environment Mapping which computes reflections in computer-synthesized environments without tracing a myriad of secondary rays. Implemented in graphics hardware, this method is fast and provides amazing reflection effects for the bubble which is constructed as a polygonal surface model.

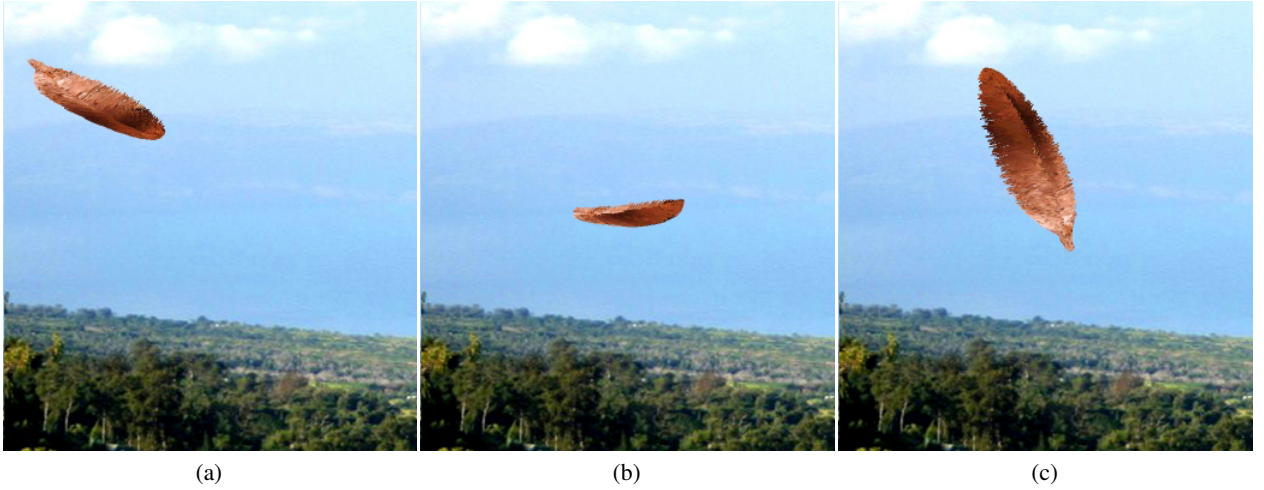
Snapshots from our single bubble interactive simulation are shown in Figures 4 and Color Plate 1. The size of the LBM lattice is modest:  $51 \times 27 \times 27$ . On the faces of the LBM flow volume we have defined out-flow boundaries to simulate an open volume. Initially the flow is quiescent. The user perturbs the flow by moving and triggering a spray-can. The can itself is implemented as a texture-mapped polygonal surface model which presents a no-slip (curved and moving) boundary to the flow. The can's nozzle is represented as a small tube approximately 6 lattice units in length and 3.6 units in diameter. The disk of nodes located half way down its length present a propeller-type inlet boundary to the volume. Texture splats are used to make the air-jet produced by the spray-can somewhat visible. They provide visual detail without requiring a high resolution grid. The splats are introduced at the nozzle when the sprayer is triggered and advected by the LBM along the flow stream lines. When their velocity decreases below a threshold value, the splats are removed. The user may "blow in the wind," from any location and orientation (by positioning and spraying the spray-can) and observe the bubble's response to the changes in the flow. In particular, the bubble may be deformed and pushed away by a nearby burst of air. The magnitude of the deformation ( $\delta$  in Equation 16) is made proportional to the gradient of the wind velocity along the direction of advection. This deformation excites the  $l = 2$  mode which subsequently decays as the bubble drifts away from the can. If the bubble drifts outside the flow volume, it pops by simply disappearing.

Snapshots of our multiple bubble animation are shown in Figures 5 and 6 and Color Plates 2 and 4. This simula-

tion demonstrates a cloud of bubbles being blown by a flow as large vortices develop. Again, the LBM lattice size is a modest  $51 \times 27 \times 27$ . We chose to make all the bubbles the same size so that the viewer may better judge their relative positions in the scene. The flow pattern results from a low speed, horizontally directed, propeller type inlet boundary defined over the entire left vertical face of the flow volume. Another smaller, but faster propeller inlet located in the center of the floor of the flow volume and directed vertically upward serves to create large vortices in the flow. A second small propeller inlet with a diameter of 4 lattice units located near the lower edge of the left face of the volume directs additional flow into the volume at an angle of 45 degrees with respect to the horizontal. This inlet also serves as the source of new bubbles during the simulation. To help the viewer visualize the time varying flow, we show streamlines originating from a plane passing through the most active flow region. Away from this plane the flow is weaker. The pink (light green) streamlines indicate upward (downward) flow and gray streamlines indicate horizontal flow shown in Color Plate 3. As seen in the animation, bubbles near the strongest flow regions show large acceleration and deformation. In less active regions the bubbles drift due to advection and slowly fall under the influence of gravity.

## 5. Feather Simulation

A feather is a branching structure developed on a spine called the *rachis*. From the rachis a number of *barbs* originate. The collection of the barbs on each side of the feather comprise two *vanes*<sup>22</sup>. Following Streit and Heidrich<sup>8</sup>, we model the shapes of the rachis and the contour of the left and right vanes with three different Bezier curves. In order to construct each individual barb, four control points are used:  $P_0$ ,  $P_1$ ,  $P_2$ , and  $P_3$ . Points  $P_0$  and  $P_3$  are located on the rachis and the vane contour. Points  $P_1$  and  $P_2$  are generated by perturbing the positions of two points lying on the line between  $P_0$  and  $P_3$  using a noise function. Finally, a texture



**Figure 7:** A sequence of snapshots taken from our feather animation. (See Color Plate 5 in the color section.)

is mapped onto the barbs to give the feather the desired color and quality.

Free feathers dancing in the wind illustrate beautiful and complex dynamics. In response to lift and drag forces they float and flutter. The dynamics of a falling feather is non-trivial not only because the forces of lift and drag couple the downward and side-to-side motions, but also because of the production and influence of vortices in the surrounding flow. Even for the simplified system of a thin, flat strip, the effort to construct faithful physical models continues to challenge researchers (see Belmonte et al.<sup>18</sup> and references therein). To capture the essential flutter dynamics for our simulation, we use the thin strip model of Belmonte et al., which includes the effects of gravity, lift, and inertial drag. The scaled equations of motion in this 2D model are given by:

$$\dot{v}_x = \frac{V^2}{Fr} [A_{\perp} \sin \phi \sin \theta - A_{\parallel} \cos \phi \cos \theta \mp 4\pi |\sin \phi| \sin \gamma] \quad (18)$$

$$\dot{v}_y = -\frac{V^2}{Fr} [A_{\perp} \sin \phi \cos \theta + A_{\parallel} \cos \phi \sin \theta \mp 4\pi |\sin \phi| \cos \gamma] - \frac{1}{Fr} \quad (19)$$

$$\dot{\omega} = \pm A_{\omega} \frac{12\omega^2}{Fr^2} - 6\pi V^2 \sin(2\phi), \quad \dot{\theta} = \omega \quad (20)$$

Here,  $\mathbf{V} = (v_x, v_y)$  represents the velocity of the feather relative to the flow with the  $\mathbf{y}$  axis pointing vertically upward. The angle  $\theta$  is that of the rachis with respect to the positive  $\mathbf{x}$  axis, and  $\gamma$  is the angle similarly defined by  $\mathbf{V}$ . The angle  $\phi = \gamma - \theta$ . The constants  $A_{\parallel}$  and  $A_{\perp}$  represent the parallel and perpendicular drag coefficients, respectively. Finally, the dimensionless quantity  $Fr$  is the *Froude* number

which represents the ratio of downward and side-to-side motion time scales. For an explanation of the sign convention we refer the reader to Tanabe and Kaneko<sup>19</sup>.

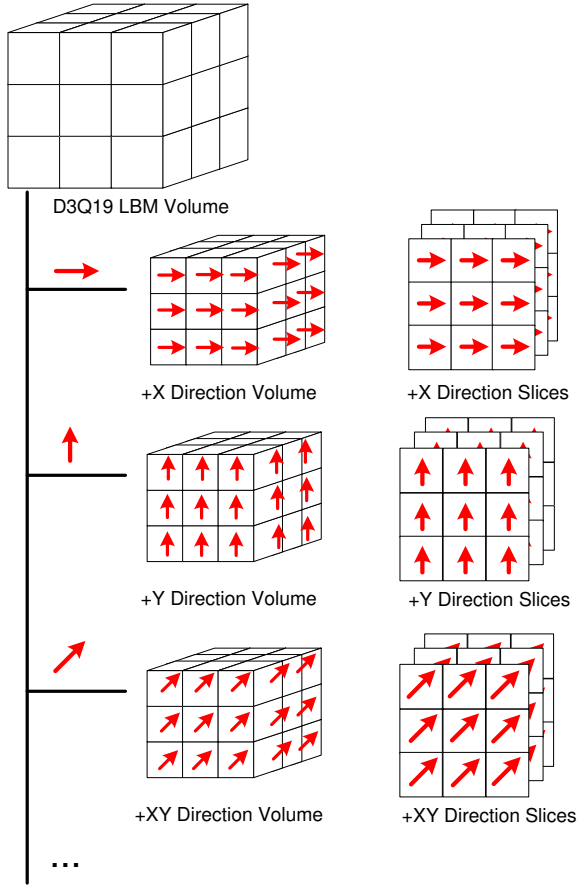
To use this 2D thin strip model in our 3D simulation we employ standard fourth-order Runge-Kutta integration within a reference frame that follows any rotation of the feather about the  $\mathbf{x}$ -axis. Such rotations may arise due to unbalanced torques exerted on the feather by the wind. We model this rotational dynamics in terms of a damped rotator. Since the thin strip model defined above assumes that the strip falls through a motionless medium, to capture the effects of a gusty wind, we simply add a term proportional to the appropriate component of the wind force to Equations 18 and 19. This force is obtained from the LBM by considering the change in the flow velocity at the center of mass of the feather over consecutive time steps.

Snapshots from our feather animation are shown in Figures 7 and Color Plate 5. A gusty wind field was produced using out-flow boundary conditions and the methods described in Section 3.4 with several additional propeller inlets defined on the left and bottom boundaries. The LBM lattice size is  $51 \times 31 \times 31$ . Although we did not include deformation dynamics in our current feather model, the feather is seen to flutter and dance in wind as it slowly falls under the influence of gravity.

## 6. Hardware Acceleration

We accelerate the computation of the flow field using the graphics hardware (GPU) as a SIMD computer in a fashion similar to that of Harris et al.<sup>20</sup> and Li et al.<sup>5</sup>. The process involves transforming the LBM nodes into stacks of 2D textures and computing the LBM equations on the rasterization pipeline of the hardware. Specifically, the packet





**Figure 8:** Packet distributions are grouped into textures according to their velocity directions.

distributions  $f_{qi}$  from each node are grouped into stacks of 2D textures according to their velocity directions as shown in Figure 8. The macroscopic density  $\rho$  and velocity  $\mathbf{u}$ , and the equilibrium distributions are stored in a similar way. The packet distribution textures then serve as input to the computation of density and velocity. The equilibrium distribution values  $f_{qi}^{eq}$  are computed in a similar fashion using the density and velocity textures.

To perform the LBM streaming step, the distribution textures must propagate in the direction of their associated velocity. For those velocity components within the slice, we simply translate the packet distribution textures in the direction of the in-slice velocity. For those velocity components orthogonal to the slice, we re-number the distribution textures to achieve propagation in the direction of the orthogonal velocity. To reduce the overhead of switching between textures, multiple textures representing packet distributions with the same velocity direction are stitched together into one larger texture. In addition to packet distribution transla-

tions, streaming on this larger texture then requires copying sub-textures to other stitched textures.

Next, we apply the boundary conditions and update the distribution textures. Implementing Mei et al.'s no-slip boundary condition on the GPU requires evaluating the boundary rule using two nodes, a real one near the boundary, such as  $\mathbf{x}_f$  shown in Figure 2, and a fictitious one,  $\mathbf{x}_b$ , located inside the boundary object. For each such pair of nodes we load two "node-sized" textures with the corresponding packet distributions, velocities, and densities as well as parameters, such as  $\Delta$  and  $\mathbf{u}_w$ . These texture pairs are then streamed through the GPU pipeline to evaluate Equation 10.

We also accelerate the Smagorinsky subgrid model on the GPU. Considering Equation 6, we see that the adjustment of the relaxation parameter,  $\tau$ , is only necessary for nodes where the nonequilibrium momentum tensor is nonzero. Fortunately, we have found that it is enough to use the subgrid model to adjust  $\tau$  in regions where the velocity is relatively large. We therefore divide the LBM nodes into blocks and down-sample the corresponding velocity textures using the frame buffer with the *max* blending function. Several passes of down-sampling are executed. On each pass, the size of the texture was halved along each dimension until every pixel contained the max velocity of the sub-texture defined by the corresponding block. These max velocity values determine whether the sub-textures are needed for the subgrid computation. Once blocks requiring sub-grid computation are identified, the hardware computation is straightforward. In our simulations, it turned out that less than 10% of nodes per time step required relaxation time adjustment. Thus the inclusion of the subgrid model did not significantly affect execution time.

We compute the force of the wind field and the object physics on the CPU. This requires accessing the velocity data calculated on the graphics hardware. We make this efficient by dividing the velocity field into blocks. Based on the object position, only the needed velocity blocks are transferred from the GPU to main memory. In general, pixel read-back to the CPU would represent a potential bottleneck for our hardware acceleration. Fortunately, Nvidia's *pixel\_data\_range* and similar extensions, allow copying the pixels in the frame buffer directly to a vertex array residing in the graphics memory without going through the AGP bus.

Because most graphics hardware supports only 8 bits per color channel, accuracy is always a concern when using graphics hardware for general computation. Fortunately, the packet distributions  $f_{qi}$ , velocities  $\mathbf{u}$ , and densities  $\rho$  in our LBM simulations tend to fall into a small numerical range which makes the use of range scaling and range separation effective. Thus, we use scale factors to map all variables into  $[-1, 1]$  or  $[0, 1]$ , taking care that all intermediate values also fall within this range. The scale factors are carefully chosen so that clamping errors do not occur while the full precision of the hardware is exploited.

While fixed point precision is less of an issue with the new generation of GPUs, such as ATI's R300 and Nvidia's NV30, which have floating point computation available throughout the fragment shader, floating point computation does require more memory and is slower than its fixed-point counterpart. Thus, even when floating point computation is available, for those simulations where speed is more important than precision, it may still be preferable to employ range scaling and use fixed point computation.

Table 1 reports running times on a 2.53GHz Pentium IV with an Nvidia GeForce 4 Ti 4600. The average calculation time of the wind field per step on the CPU (LBM on CPU) and accelerated on the GPU (LBM on GPU) are shown in the table. The computation of the soap bubble and feather dynamics, the evaluation of the body forces on the bubble and feather and the object rendering are all implemented on the CPU and are included in the last column of Table 1.

**Table 1:** Running time per frame measured in milliseconds.

| Simulation | Lattice Size             | LBM on GPU | LBM on CPU | Rendering & Object Physics on CPU |
|------------|--------------------------|------------|------------|-----------------------------------|
| Bubble     | $51 \times 27 \times 27$ | 10.1       | 93         | 16                                |
| Bubbles    | $51 \times 27 \times 27$ | 10.1       | 93         | 47                                |
| Feather    | $51 \times 31 \times 31$ | 14.7       | 125        | 50                                |

## 7. Conclusions and Future Work

In this paper, we animated free, light-weight and mildly deformable objects blowing in the wind. To model the flow field, we used the LBM because it is easy to work with, can accommodate complex and moving boundaries, and can achieve real-time speed via graphics hardware acceleration. We demonstrated our approach with detailed simulations of soap bubbles and a feather. For the bubble we used the lowest order, physically relevant, spherical harmonic to represent deformation and oscillation. A thin strip model was used to represent the dynamics of the free falling feather. We coupled the flow to these models by considering the forces arising due to changes in the flow velocity in the reference frame of the moving object. We demonstrated how interesting flows can be created in an intuitive way by simply placing obstacles in the flow and defining inlets and other boundary conditions. We also implemented a hardware accelerated version of the Smagorinsky subgrid model to achieve faster flows and the no-slip boundary condition rule. Our results

illustrate the interesting dynamics that emerge from the coupling of a time-varying flow to light-weight and deformable object models.

Of course, much more can be achieved. In particular, we would like to employ more general object models, such as mass-spring and finite element models, and explore techniques for computing their dynamics in another pass on the graphics hardware. In the future, we will incorporate the perturbations induced in flow by the objects. Including this back-coupling will make possible the inclusion of heavy objects and the use of the momentum exchange method for computing the flow forces. Finally we will adopt the new generation of graphics hardware on which floating point operations are possible. Exploring the tradeoffs between speed and precision will provide valuable insights on the LBM method and its potential for both visual and scientific simulation. Moreover, using Nvidia's *pixel\_data\_range* or similar extension to copy pixels in the frame buffer directly to a vertex array residing in the graphics memory will enable us to perform both simulation and rendering completely inside the GPU.

## Acknowledgments

This work is partially supported by ONR grant N000140110034 and NSF grant IIS-0097646. We would like to thank Michael Ashikhmin for helpful discussions and insightful comments.

## References

1. J. Stam. Stable Fluids. *ACM Computer Graphics (Proc. of SIGGRAPH '99)*, 121–128, 1999.
2. N. Foster and D. Metaxas. Modeling the Motion of a Hot, Turbulent Gas. *ACM Computer Graphics (Proc. of SIGGRAPH '97)*, 181–188, 1997.
3. R. Fedkiw and J. Stam and H. W. Jensen. Visual Simulation of Smoke. *ACM Computer Graphics (Proc. of SIGGRAPH '01)*, 15–22, 2001.
4. X. Wei and W. Li and K. Mueller and A. Kaufman. Simulating Fire with Textured Splats. *IEEE Visualization*, 227–234, 2002.
5. W. Li and X. Wei and A. Kaufman. Implementing Lattice Boltzmann Computation on Graphics Hardware. *The Visual Computer* 2003, to appear.
6. X. Wei and W. Li and K. Mueller and A. Kaufman. The Lattice Boltzmann Method for Gaseous Phenomena. *IEEE Transaction on Visualization and Computer Graphics* 2003, to appear.
7. S. Hou and J. Sterling and S. Chen and G.D. Doolen. A lattice Subgrid Model for High Reynolds Number Flows. *Fields Institute Communications*, 6:151–166, 1996.

8. L. Streit and W. Heidrich A Biologically-Parameterized Feather Model. *Computer Graphics Forum (Eurographics'02 Proc.)*, **21**(3):565–573, 2002.
9. D. A. Wolf-Gladrow. *Lattice gas cellular automata and lattice Boltzmann models: an introduction*. Springer-Verlag, 2000.
10. S. Chen and G. D. Doolean. Lattice Boltzmann Method for Fluid Flows. *Annu. Rev. Fluid Mech.*, **30**:329–364, 1998.
11. R. Mei and W. Shyy and D. Yu and L.-S. Luo. Lattice Boltzmann method for 3-D Flows with curved boundary. *Journal of Comp. Phys.*, **161**:680–699, 2000.
12. N.S. Martys and J.G. Hagedorn and D. Goujon and J.E. Devaney. Large Scale Simulations of Single and Multi-Component Flow in Porous Media. *Proc. SPIE*, **3772**:205–213, 1999.
13. R. Mei and D. Yu and W. Shyy and L.-S. Luo. Force evaluation in the lattice Boltzmann method involving curved geometry. *Phys. Rev. E*, **65**, 2002.
14. R. Durikovic. Animation of soap bubble dynamics, cluster formation and collision. *Comp. Graphics Forum*, **20**(3):67–75, 2001.
15. A. Glassner. Soap bubbles. *IEEE Computer Graphics and Applications*, **20**(5):76–84, **20**(6):99–109, 2000.
16. I. Icart and D. Arques. An Approach to geometrical and optical simulation of soap froth. *Computers and Graphics*, **23**(3):405–418, 1999.
17. C. A. Miller and L. E. Scriven. The oscillations of a fluid droplet immersed in another fluid. *J. Fluid Mech.*, **32**(3):417–435, 1968.
18. A. Belmonte and H. Eisenberg and E. Moses. From Flutter to Tumble: Inertial Drag and Froude Similarity in Falling Paper. *Physical Review Letters*, **81**(2):345–348, 1998.
19. Y. Tanabe and K. Kaneko. Behavior of a Falling Paper. *Physical Review Letters*, **73**(10), 1994.
20. M. J. Harris and G. Coombe and T. Scheuermann and A. Lastra. Physically-based visual simulation on graphics hardware. *Proceedings of the Conference on Graphics Hardware*, 109–118, 2002.
21. J. Wejchert and D. Haumann. Animation Aerodynamics. *ACM Computer Graphics (Proc. of SIGGRAPH '91)*, 19–22, 1991.
22. Y. Chen and Y. Xu and B. Guo and H. Shum. Modeling and Rendering of Realistic Feathers. *ACM Computer Graphics (Proc. of SIGGRAPH '02)*, 630–636, 2002.
23. J. Stam. Stochastic Dynamics: Simulating the Effects of Turbulence on Flexible Structures. *Computer Graphics Forum*, **16**(3):159–164, 1997.
24. M. Shinya and A. Fournier. Stochastic Motion - Motion Under the Influence of Wind. *Computer Graphics Forum*, **11**(3):C119–128, 1992.
25. F. Perbet and M.-P. Cani. Animating Prairies in Real-Time. *ACM Symposium on Interactive 3D Graphics*, 103–110, 2001.
26. L. M. Reissel and D. K. Pai. Modelling Stochastic Dynamical Systems for Interactive Simulation. *Computer Graphics Forum*, **20**(3):339–348, 2001.
27. S. Hadap and N. Magnenat-Thalmann. Modelling Dynamic Hair as a Continuum. *Computer Graphics Forum*, **20**(3):329–338, 2001.
28. G. G. McNamara and G. Zanetti. Use of the Boltzmann Equation to simulate Lattice-Gas Automata. *Physical Review Letters*, **61**:2332–2335, 1988.
29. R. S. Maier and R. S. Bernard and D. W. Grunau. Boundary conditions for the lattice Boltzmann method. *Physics of Fluids*, **8**(7):1788–1801, 1996.

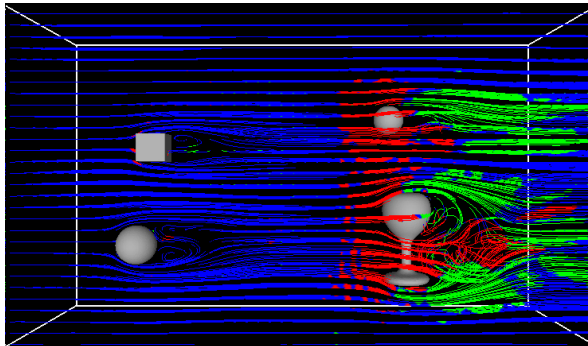




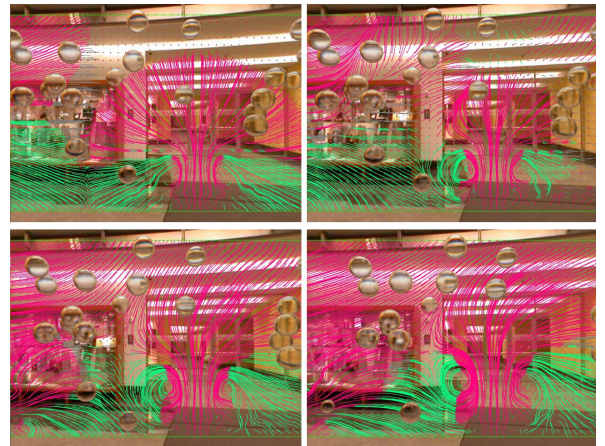
Color Plate 1: Soap bubble deformation and dynamics in response to the spray from the green air-can.



Color Plate 2: Snapshots from our multi bubble animation, as we move the viewpoint position from right to left.



Color Plate 3: Streamlines originating from a vertical plane cuts through the most active flow area. Red (green) lines indicate flow toward (away from) the viewer, and blue lines indicate the flow in the plane.



Color Plate 4: Streamlines originating from a plane cutting through the most active flow area. Pink (light green) lines indicate upward (downward) flow. Gray lines indicate horizontal flow.



Color Plate 5: A sequence of snapshots taken from our feather animation.